Selamat datang

WPU Ramadhan Camp

Kajian React.js Dari Dasar Hingga Mahir



sponsored by:



WPU Ramadhan Camp

Kajian React JS Dari Dasar Sampai Mahir

Rules Bootcamp

- Masuk ke zoom beberapa menit sebelum waktu yang dijadwalkan untuk menghindari ketertinggalan materi
- Gunakan Virtual Background WPU Ramadhan Camp
- Jika memungkinkan, aktifkan kamera untuk meningkatkan interaksi dan partisipasi
- Matikan mikrofon Anda (mode senyap) saat tidak berbicara untuk menghindari gangguan suara latar
- Apabila memiliki pertanyaan kita menyediakan slido supaya apabila teman-teman punya pertanyaan yang sama tinggal upvote pertanyaan yang lain yang sudah diajukan oleh rekan yang lain

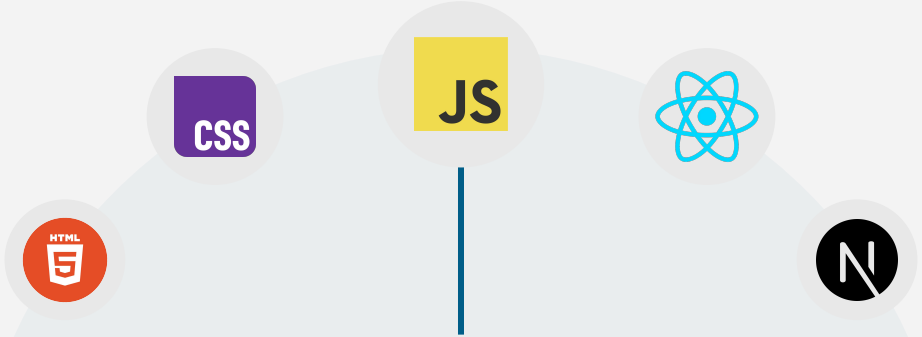
pertanyaan

Join at

slido.com

#3508 753

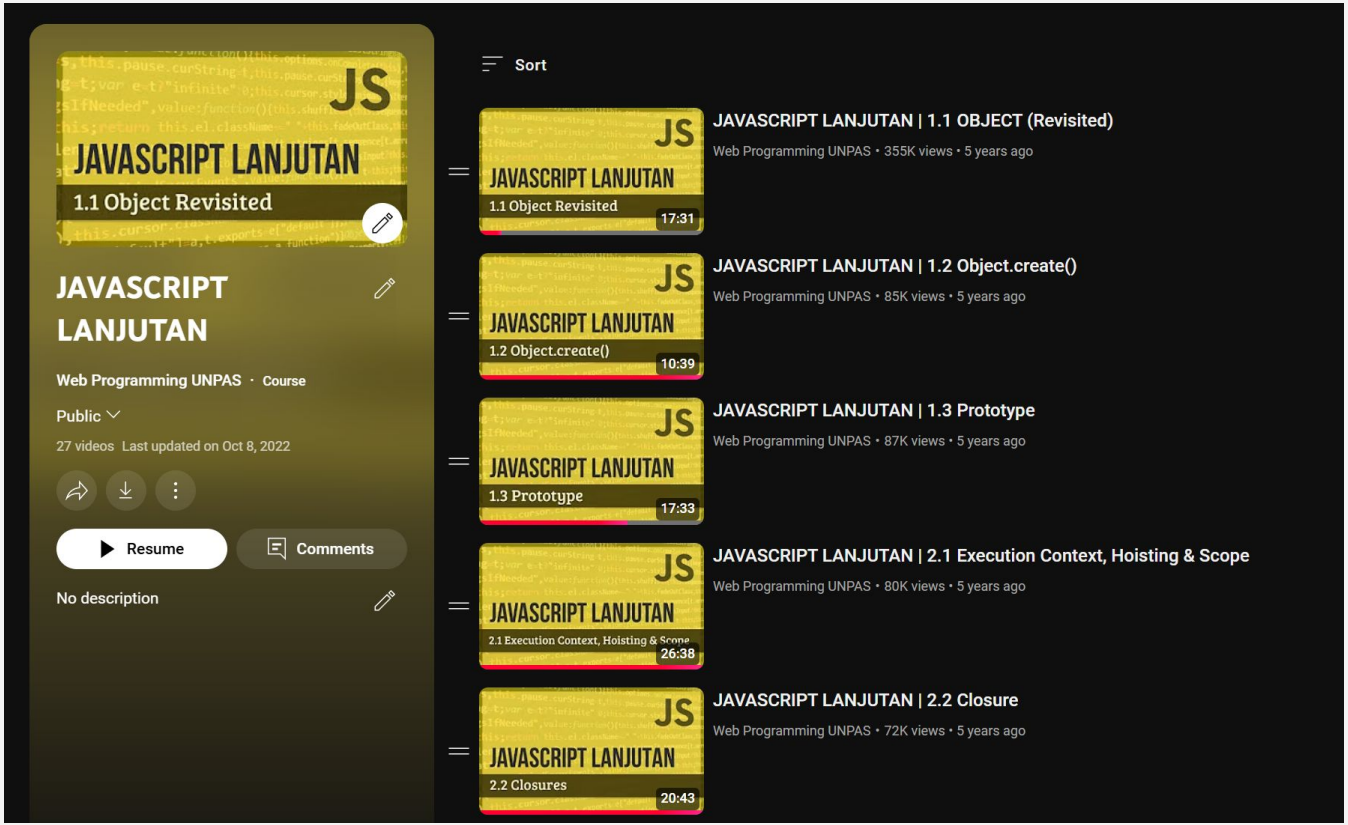


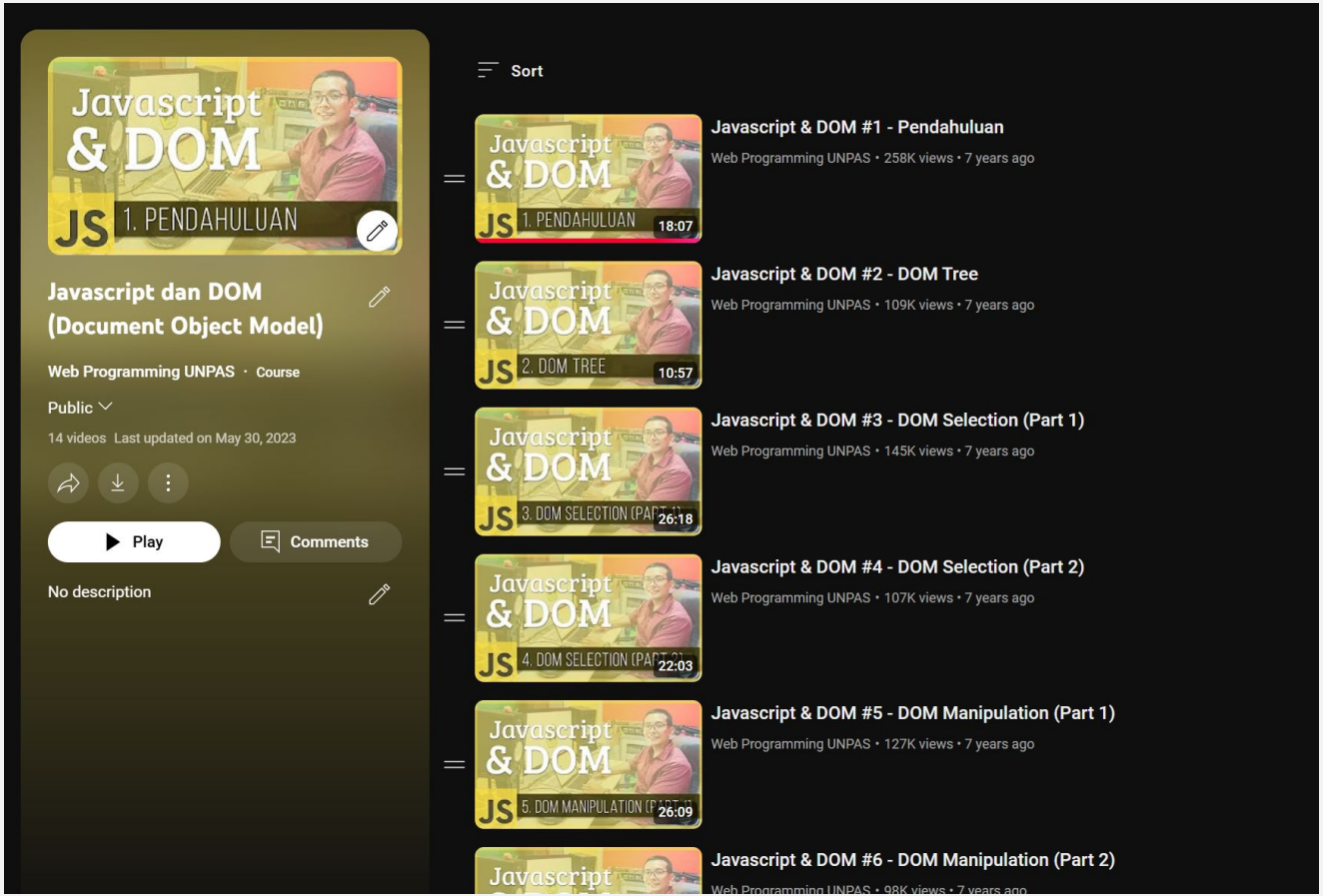


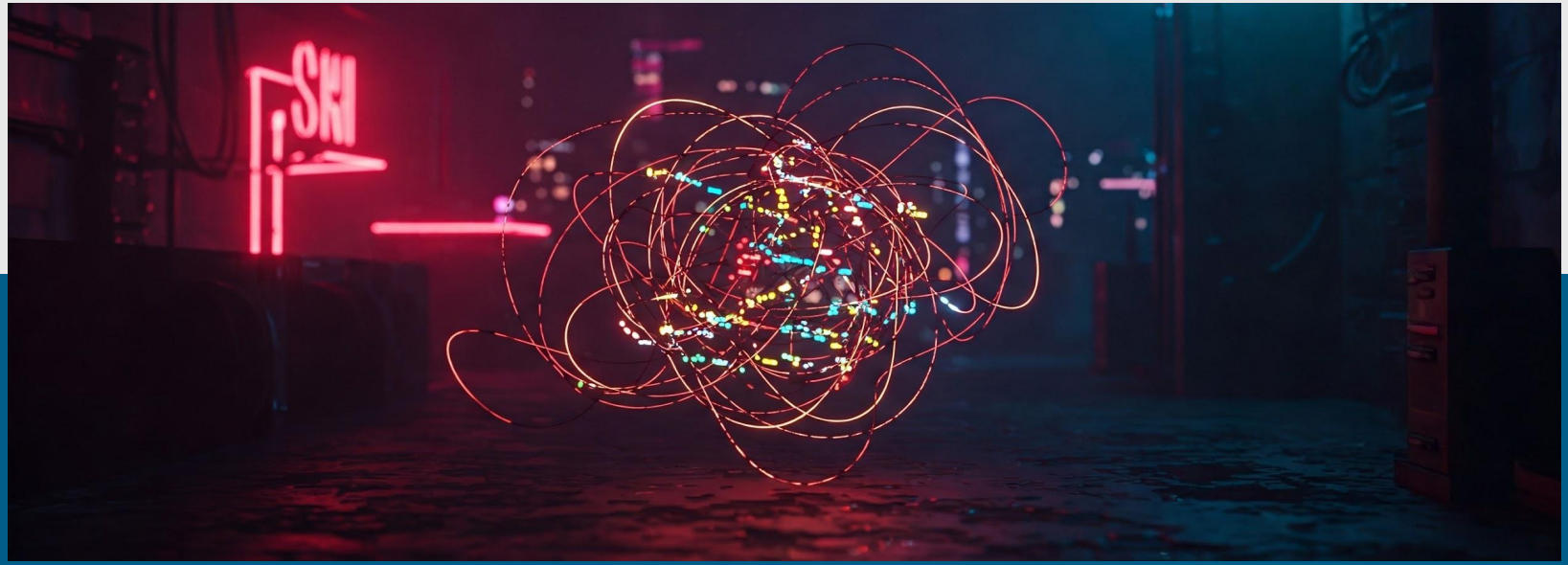
Case Study

Sandhika Galih

pre-requisite


JS
JAVASCRIPT LANJUTAN
1.1 Object Revisited
JAVASCRIPT LANJUTAN
Web Programming UNPAS · Course
Public
27 videos Last updated on Oct 8, 2022
Resume
Comments
No description
Sort
JAVASCRIPT LANJUTAN | 1.1 OBJECT (Revisited)
Web Programming UNPAS • 355K views • 5 years ago
17:31
JAVASCRIPT LANJUTAN | 1.2 Object.create()
Web Programming UNPAS • 85K views • 5 years ago
1.2 Object.create()
10:39
JAVASCRIPT LANJUTAN | 1.3 Prototype
Web Programming UNPAS • 87K views • 5 years ago
1.3 Prototype
17:33
JAVASCRIPT LANJUTAN | 2.1 Execution Context, Hoisting & Scope
Web Programming UNPAS • 80K views • 5 years ago
2.1 Execution Context, Hoisting & Scope
26:38
JAVASCRIPT LANJUTAN | 2.2 Closure
Web Programming UNPAS • 72K views • 5 years ago
2.2 Closures
20:43


Javascript & DOM
JS 1. PENDAHULUAN
Javascript dan DOM (Document Object Model)
Web Programming UNPAS · Course
Public
14 videos Last updated on May 30, 2023
Play
Comments
No description
Sort
Javascript & DOM #1 - Pendahuluan
Web Programming UNPAS • 258K views • 7 years ago
18:07
Javascript & DOM #2 - DOM Tree
Web Programming UNPAS • 109K views • 7 years ago
2. DOM TREE
10:57
Javascript & DOM #3 - DOM Selection (Part 1)
Web Programming UNPAS • 145K views • 7 years ago
3. DOM SELECTION
26:18
Javascript & DOM #4 - DOM Selection (Part 2)
Web Programming UNPAS • 107K views • 7 years ago
4. DOM SELECTION
22:03
Javascript & DOM #5 - DOM Manipulation (Part 1)
Web Programming UNPAS • 127K views • 7 years ago
5. DOM MANIPULATION
26:09
Javascript & DOM #6 - DOM Manipulation (Part 2)



01 Callback Hell

callback **hell?**

masalah yang terjadi ketika ketika kita memiliki banyak operasi **asynchronous** yang saling **bergantung** satu sama lain.

contoh

```js
function bacaFile(namaFile, callback) {
  setTimeout(() => {
    const data = 'Data dari file';
    console.log(`Data dari file ${namaFile} dibaca.`);
    callback(data);
  }, 1000);
}

function prosesData(data, callback) {
  setTimeout(() => {
    const hasil = data.toUpperCase();
    console.log('Data diproses:', hasil);
    callback(hasil);
  }, 1000);
}

function tampilkanHasil(hasil, callback) {
  setTimeout(() => {
    console.log('Hasil pemrosesan:', hasil);
    callback();
  }, 1000);
}
```

```js
// Memanggil fungsi-fungsi dengan callback
bacaFile('data.txt', (data) => {
  prosesData(data, (hasil) => {
    tampilkanHasil(hasil, () => {
      console.log('Selesai!');
    });
  });
});
```

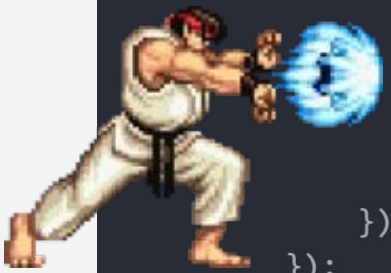
Callback Hell

Callback Hell

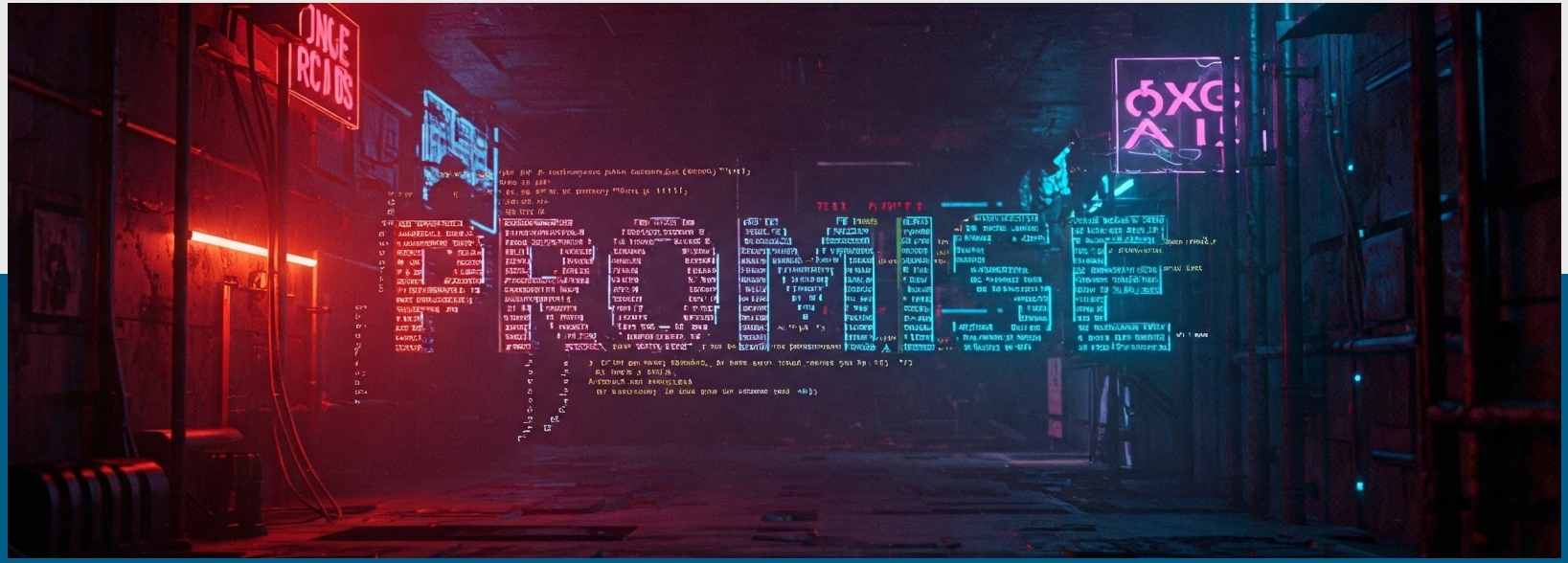
- Problem pada indentasi code (***pyramid of doom***)
- Sulit **membaca** code
- Code **sulit dikelola** dan **di-debug**
- ***Inversion of Control***: kendali pada fungsi yang kita buat, diserahkan pada fungsi lain yang memanggilnya

pyramid of **doom**?

```
asyncFunc('something', function(err, data) {
    if(err) return console.log('an error occurs:', err);
    asyncFunc('something', function(err, data) {
        if(err) return console.log('an error occurs:', err);
        asyncFunc('something', function(err, data) {
            if(err) return console.log('an error occurs:', err);
            asyncFunc('something', function(err, data) {
                if(err) return console.log('an error occurs:', err);
                asyncFunc('something', function(err, data) {
                    if(err) return console.log('an error occurs:', err);
                    // do the final action
                });
            });
        });
    });
});
```



solusi?



02 Promise

Promise

- "***Janji***" yang diberikan oleh sebuah operasi yang mungkin **belum selesai** saat itu
- Janji ini akan memberikan kabar (hasil) di **masa depan**
- Bentuknya adalah **objek** yang mewakili **hasil akhir** dari sebuah operasi **asinkron**

```js
const janji = new Promise((resolve, reject) => {
  setTimeout(() => {
    const berhasil = false;
    if (berhasil) {
      resolve('Janji ditepati!'); // Operasi berhasil
    } else {
      reject('Janji dibatalkan!'); // Operasi gagal
    }
  }, 2000);
});
```

Membuat promise

```js
janji
  .then((hasil) => {
    console.log(hasil); // "Janji ditepati!"
  })
  .catch((alasan) => {
    console.error(alasan); // "Janji dibatalkan!"
  });
```

Menjalankan promise

Komponen pada Promise (1)

- **Constructor**: Membuat object Promise, berisi 2 parameter berupa function yaitu **resolve()** dan **reject()**.
- **Executor** : Fungsi yang bertanggung jawab untuk memulai operasi asinkron dan memanggil **resolve()** atau **reject()** ketika operasi selesai.
  - **"resolve()"** : Fungsi yang dipanggil untuk menandai Promise sebagai "fulfilled".
  - **"reject()"** : Adalah fungsi yang dipanggil untuk menandai Promise sebagai "rejected".
- **PromiseReaction** : Daftar internal dari reaksi yang terdaftar untuk Promise
  - "**.then()**" : dipanggil ketika PromiseState berubah menjadi "fulfilled".
  - "**.catch()**" : dipanggil ketika PromiseState berubah menjadi "rejected".

Komponen pada Promise (2)

- **[[PromiseState]]** : Status internal dari Promise yang menunjukkan keadaannya
  - **"pending"** : Keadaan awal, artinya Promise sedang dalam proses dan belum selesai.
  - **"fulfilled"** : Artinya Promise telah berhasil diselesaikan.
  - **"rejected"** : Artinya Promise gagal diselesaikan.
- **[[PromiseResult]]** : Nilai internal yang terkait dengan hasil Promise
  - Jika PromiseState berisi "**fulfilled**", PromiseResult akan berisi **nilai hasil** dari operasi asinkron.
  - Jika PromiseState berisi "**rejected**", PromiseResult akan berisi alasan mengapa operasi gagal (biasanya **objek Error**).

Constructor

```
const janji = new Promise((resolve, reject) => {
  setTimeout(() => {
    const berhasil = false;
    if (berhasil) {
      resolve('Janji ditepati!'); // Operasi berhasil
    } else {
      reject('Janji dibatalkan!'); // Operasi gagal
    }
  }, 2000);
});

janji
  .then((hasil) => {
    console.log(hasil); // "Janji ditepati!"
  })
  .catch((alasan) => {
    console.error(alasan); // "Janji dibatalkan!"
  });
```

Executor

```
janji: Promise
  [[Prototype]]: Promise
  [[PromiseState]]: "fulfilled"
  [[PromiseResult]]: "Janji ditepati!"
```

Promise reaction

mengubah **callback** menjadi **promise**

```
function bacaFile(namaFile, callback) {
  setTimeout(() => {
    const data = 'Data dari file';
    console.log(`Data dari file ${namaFile} dibaca.`);
    callback(data);
  }, 1000);
}

function prosesData(data, callback) {
  setTimeout(() => {
    const hasil = data.toUpperCase();
    console.log('Data diproses:', hasil);
    callback(hasil);
  }, 1000);
}

function tampilkanHasil(hasil, callback) {
  setTimeout(() => {
    console.log('Hasil pemrosesan:', hasil);
    callback();
  }, 1000);
}
```

```
// Memanggil fungsi-fungsi dengan callback
bacaFile('data.txt', (data) => {
  prosesData(data, (hasil) => {
    tampilkanHasil(hasil, () => {
      console.log('Selesai!');
    });
  });
});
```

```
function bacaFilePromise(namaFile) {
  return new Promise((resolve) => {
    setTimeout(() => {
      const data = 'Data dari file';
      console.log(`Data dari file ${namaFile} dibaca.`);
      resolve(data);
    }, 1000);
  });
}

function prosesDataPromise(data) {
  return new Promise((resolve) => {
    setTimeout(() => {
      const hasil = data.toUpperCase();
      console.log('Data diproses:', hasil);
      resolve(hasil);
    }, 1000);
  });
}

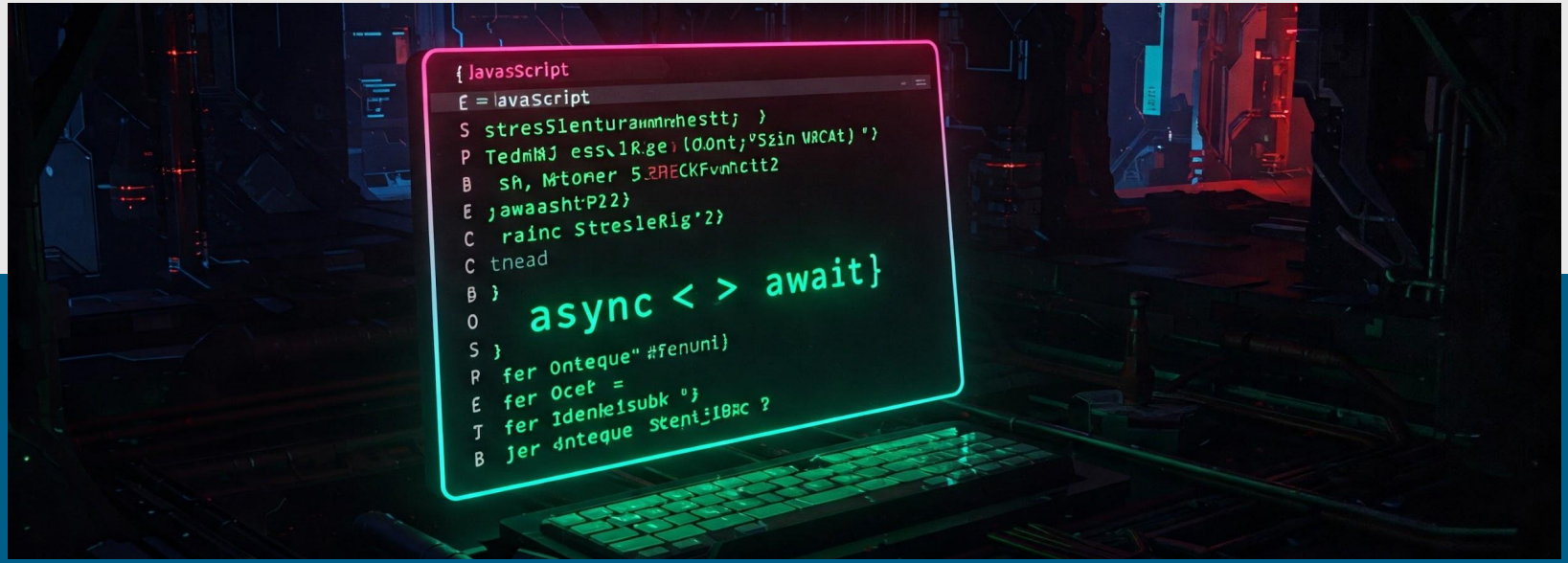
function tampilkanHasilPromise(hasil) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log('Hasil pemrosesan:', hasil);
      resolve();
    }, 1000);
  });
}
```

```
bacaFilePromise('data.txt')
  .then(prosesDataPromise)
  .then(tampilkanHasilPromise)
  .then(() => {
    console.log('Selesai dengan Promise!');
  })
  .catch((error) => {
    console.error('Terjadi kesalahan:', error);
  });
```

promise memang menjadi **solusi**

tapi,

alangkah **lebih baik** jika menulis program
asynchronous secara **synchronous**



03 Async & Await

Async & Await

- Fitur dalam Javascript yang membuat kode **asynchronous** terlihat dan terasa seperti kode **synchronous**.
- Menggunakan keyword **async** untuk mendefinisikan fungsi asinkron dan **await** untuk menghentikan eksekusi fungsi hingga sebuah **Promise** diselesaikan.
- Bertujuan untuk mempermudah **keterbacaan kode** dan **penanganan kesalahan**.
- Menggunakan **try** dan **catch** untuk **error handling**

```
1  const p = new Promise((resolve, reject) => {
2    setTimeout(() => {
3      resolve('nilai hasil promise ketika resolved');
4    }, 2000);
5  });
6
7  async function getResult() {
8    console.log(await p);
9  }
10
11 getResult();
```

Terlihat seperti kode synchronous

mengubah **promise** menjadi **async/await**

```
bacaFilePromise('data.txt')
  .then(prosesDataPromise)
  .then(tampilkanHasilPromise)
  .then(() => {
    console.log('Selesai dengan Promise!');
  })
  .catch((error) => {
    console.error('Terjadi kesalahan:', error);
  });
```



```
async function handleFileAsync(file) {
  try {
    const bacaFile = await bacaFilePromise(file);
    const prosesData = await prosesDataPromise(bacaFile);
    await tampilkanHasilPromise(prosesData);
    console.log('Selesai dengan Promise!');
  } catch (e) {
    console.error('Terjadi kesalahan:', e);
  }
}

const file = 'data.txt';
handleFileAsync(file);
```

sekarang mari kita **implementasikan**
semuanya ke **studi kasus**

tapi **sebelumnya**



04 fetch()

method pada **Web API** untuk mengambil **resources** dari jaringan, dan mengembalikan **promise** yang selesai (***fullfilled***) ketika ada **response** yang tersedia.

```
fetch(resources, <option>);
```



Resources

- **URL**
  - Alamat dari sumber yang akan kita ambil
- **Request Object**
  - Hasil yang dikembalikan dari permintaan ke API lain

Options konfigurasi tambahan dalam bentuk object

- method
- headers
- body
- mode
- cache
- referrer
- referrerPolicy
- integrity
- keepalive
- signal

Response dalam bentuk **property**

- headers
- ok
- redirected
- status
- statusText
- type
- url
- body

Response dalam bentuk **method**

- arrayBuffer()
- blob()
- formData()
- json()
- text()

```
fetch('https://api.github.com/users/sandhikagalih');
```

```
fetch('https://api.github.com/repos/facebook/react/issues', {
  method: 'POST',
  headers: {
    'Authorization': 'token YOUR_GITHUB_TOKEN',
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    title: 'Isu baru dari Fetch API',
    body: 'Deskripsi isu...',
    labels: ['bug', 'enhancement'],
  }),
});
```

```
fetch('https://api.github.com/users/sandhikagalih');
```

mengembalikan **promise**

jadi harus menggunakan **then()** untuk mendapatkan data nya

```
1  fetch('https://api.github.com/users/sandhikagalih')
2    .then((res) => res.json())
3    .then((user) => {
4      console.log(`Nama : ${user.name}`);
5      console.log(`Jumlah Repo : ${user.public_repos}`);
6    });
```

mengembalikan promise

mengembalikan promise

```
Nama : Sandhika Galih
Jumlah Repo : 88
```



05 DOM (Document Object Model)

DOM adalah antarmuka pemrograman
untuk **HTML**, yang merepresentasikan **halaman web**, sehingga
program dapat **mengubah** dan **memanipulasi** strukturnya.

DOM

- API yang merepresentasikan dokumen web
- DOM membuat seluruh komponen dari halaman web dapat diakses & dimanipulasi
- Komponen
  - Elemen HTML
  - Atribut
  - Text
  - dll
- DOM dapat dimanipulasi (dibuat, diubah, dihapus) menggunakan javascript

DOM

- DOM Selection
- DOM Manipulation
- DOM Events
- DOM Traversal

DOM Selection

Method	Hasil
`getElementById()`	element
`getElementsByTagName()`	HTMLCollection
`getElementsByClassName()`	HTMLCollection
`querySelector()`	element
`querySelectorAll()`	nodeList

DOM Manipulation

- Manipulasi Element
- Manipulasi Node

Manipulasi Element

- element.innerHTML
- element.style.<property>
- element.setAttribute()
- element.classList
- ...

Manipulasi Node

- document.createElement()
- document.createTextNode()
- node.appendChild()
- node.insertBefore()
- parentNode.append(), parentNode.prepend()
- ...

DOM Events

- Event Handler
- Event Listener

Event Handler

- on<event>

```
<button onclick="tampilkanPesan()">Klik Saya</button>
<p id="output"></p>

<script>
    function tampilkanPesan() {
        document.getElementById('output').textContent = 'Tombol telah diklik!';
    }
</script>
```

Event Listener

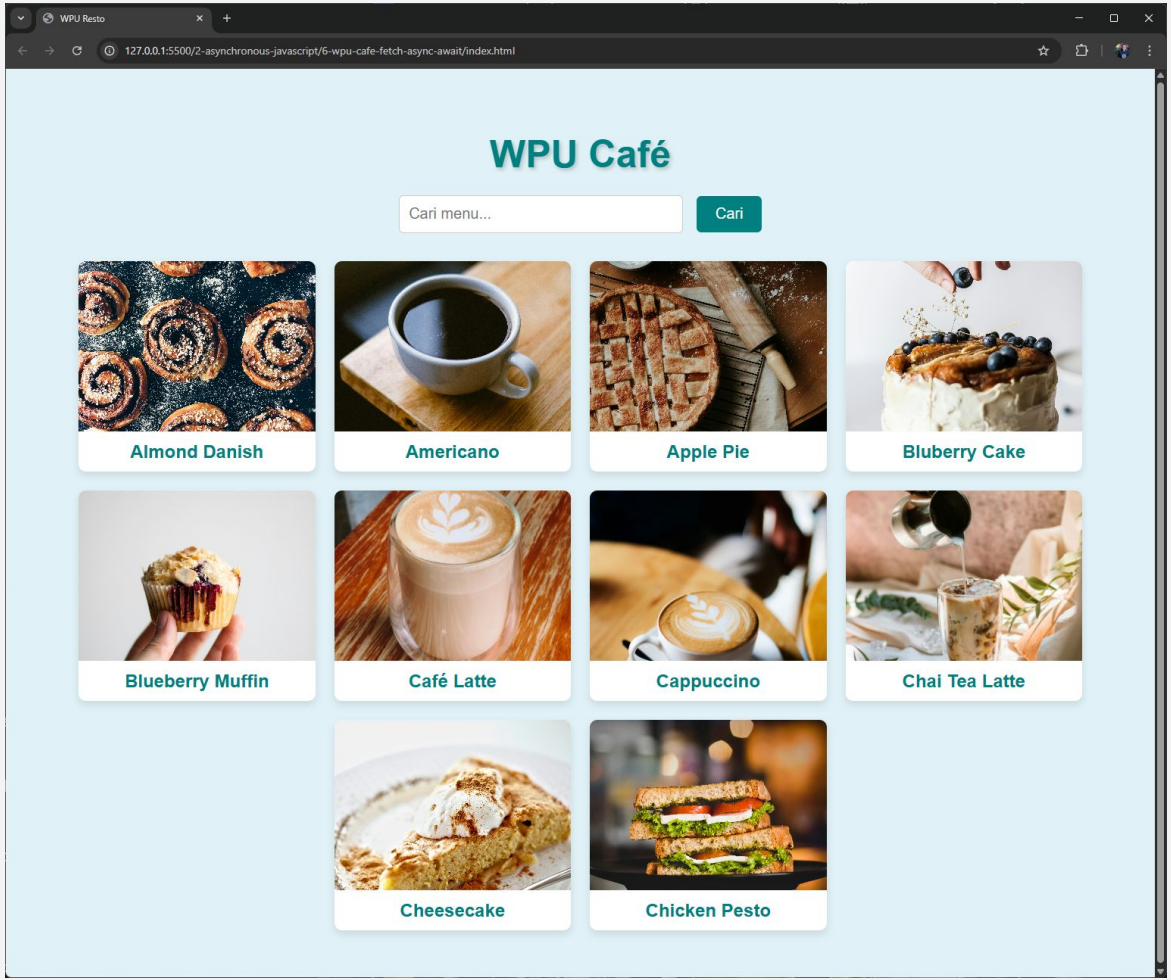
- addEventListener

```
<button id="myButton">Klik Saya</button>
<p id="output"></p>

<script>
  const button = document.getElementById('myButton');
  const output = document.getElementById('output');
  button.addEventListener('click', function () {
    output.textContent = 'Tombol telah diklik!';
  });
</script>
```

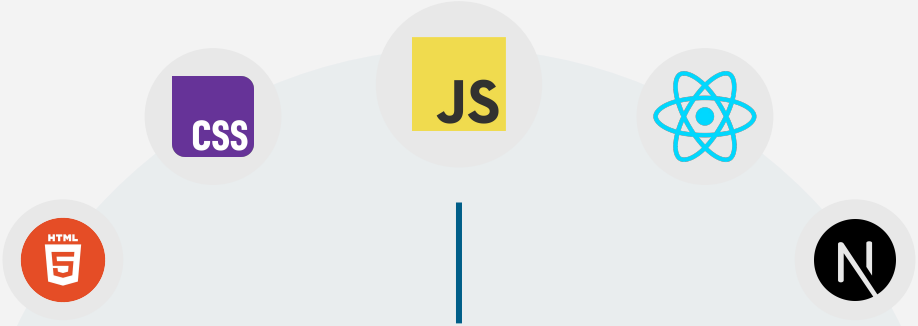


06 Case Study


WPU Resto
127.0.0.1:5500/2-asynchronous-javascript/6-wpu-cafe-fetch-async-await/index.html
WPU Café
Cari menu...
Cari
Almond Danish
Americano
Apple Pie
Bluberry Cake
Blueberry Muffin
Café Latte
Cappuccino
Chai Tea Latte
Cheesecake
Chicken Pesto

github.com/wpucourse/bootcamp-ramadhan-2025

let's **code**!



Terima kasih!

wpucourse.id

@sandhikagalih